How Short Can a Branchless Sort Be? Certified Optimality of AlphaDev's sort3 and sort4 Kernels, and What Stands in the Way for sort5

Zhiyang Chen

October 2026

Abstract

AlphaDev (Nature 2023) found x86 kernels that sort 3, 4 and 5 integers in 17, 28 and 42 instructions. We ask whether shorter kernels exist. We state a precise model — the instructions `mov`, `cmp`, `cmovl`, `cmovg`, `cmovle`, `cmovge` on signed 32-bit values, inputs and outputs in memory, every instruction counted, unbounded registers, correctness for all inputs (with ties) and all initial register and flag values — and reduce the question to a finite search through a chain of lemmas, each with a paper proof and a Lean 4 proof. Results: (i) sort3 needs 17 instructions. We give ten independent proofs, among them LRAT certificates checked by the verified checker `cake_lpr`, a Lean theorem that uses only the Lean kernel and the three standard axioms, and a Rocq proof. (ii) sort4 needs 28 instructions, by an exhaustive search over 1.39×10^9 abstract states with reviewed pruning rules; its 123 GB certificate is checked in Lean up to the bound $k \leq 17$. (iii) For sort5 we find no kernel shorter than 42. We find a second kernel family with 8 instead of 9 comparisons, show that no pure min/max kernel of length ≤ 41 exists and that garbage registers never help, and isolate the remaining open case. (iv) The model matters: if `cmp` and `cmov` may read memory (legal on x86, excluded by AlphaDev), the kernels fit in 7, 15, 26 and 39 instructions for $n = 2, \ldots, 5$.

1 Introduction

Mankowitz et al. [1] used deep reinforcement learning to find short branchless sorting kernels for 3, 4 and 5 elements, of length 17, 28 and 42 (the human-written sorting networks need 18, 28 and 46). The kernels are now in LLVM libc++. AlphaDev reports a brute-force argument that no sort3 kernel is shorter than 17 (about 10^{32} programs, with pruning), but gives no certificate and no exact model, and makes no claim for sort4 and sort5. Ullrich and Hack [4] enumerate sorting kernels in a register-only model and report optimal lengths 11 ($n = 3$) and 20 ($n = 4$); their model has one scratch register, only `cmovl`/`cmovg`, distinct inputs only, no reads of uninitialised registers, and outputs fixed in the input registers, and there is no certificate. A lower bound in that model does not transfer to AlphaDev's setting (Section 3). No lower bound for $n = 5$ was known.

Contributions. (1) A precise model of AlphaDev's setting, including memory and the exact counting (Section 3). (2) A chain of soundness lemmas from x86 semantics to a finite search problem (Section 4); one of them, the *snapshot lemma*, shows that the natural register model ("outputs in the input registers at the end") is too strict and gives the exact replacement. (3) Two SAT encodings with CEGIS and symmetry breaking, two enumerators, and verified checkers in Lean and Rocq (Section 5). (4) sort3 = 17 with ten independent proofs, including a kernel-only Lean theorem; sort4 = 28 by exhaustive search (Section 6). (5) A large discovery effort for

sort5 with exact scopes, two new structural lemmas and the remaining open case (Section 7). (6) Side results: shorter kernels with memory operands, and kernels with a shorter critical path at AlphaDev's length.

2 Background: AlphaDev's sorting kernels

The AssemblyGame. AlphaDev [1] builds a program one x86 instruction at a time. Each step appends one instruction; the state holds the program and the register and memory contents on a fixed set of test inputs. The reward combines correctness on the test inputs with a latency term. For the fixed-length sorts the latency term is the program length: "all algorithmic variants optimize for algorithm length as algorithm length and latency are highly correlated in the conditional branchless setting" (Methods); the reported Pearson correlation of length and measured latency is 1.00, 1.00, 0.98 for sort3, sort4, sort5 (SI, Table B.5). Correctness is tested on all weak orderings of the input ("ordered Bell numbers", SI App. F), that is 13, 75 and 541 inputs for $n = 3, 4, 5$. The kernels sort a buffer of n `int` values in place.

Reported lengths. Table 1a of [1] reports the lengths in Table 1. The human benchmark is "sorting networks" (Main text, Fixed sorting algorithms). The kernels in the public repository [2] and in LLVM libc++ [3] have lengths 17, 28 and *43*: "For sort 5, we used a 43 length algorithm discovered by AlphaDev, as it led to a more efficient C++ implementation" (Main text, Libc++ sort patch). The 42-instruction kernel is listed in the Supplementary Information, App. G (`Sort5_42`), with eleven further kernels (sort3: 17–20, sort4: 28–31, sort5: 42–45). We transcribed all of them and checked them on all inputs and on real hardware (Section 3).

	AlphaDev	human benchmark	source of the AlphaDev kernel
sort3	17	18	repo `Sort3AlphaDev`; SI `Sort3_17`
sort4	28	28	repo `Sort4AlphaDev`; SI `Sort4_28`
sort5	42	46	SI `Sort5_42` (repo and libc++: 43)

Table 1: Lengths in Table 1a of [1]. Length counts every instruction, loads and stores included (Section 3).

Where the savings come from. AlphaDev describes two local rewrites of sorting-network code. The *swap move* uses an order relation that an earlier comparator guarantees ($B \leq C$): the network computes $\min(A, B, C)$, but $\min(A, B)$ is enough; this removes one instruction (Fig. 3a–c, Table 2a). The *copy move* (sort8 only) uses $D \geq \min(A, C)$ (Table 2b). Table 2 shows the instruction mix: AlphaDev's sort3 and sort5 save register-to-register `mov`s (2 instead of 3, 5 instead of 9) and keep the number of `cmp`s of the optimal network (3, 5, 9); sort4 has exactly the mix of the 5-comparator network.

AlphaDev's own optimality claim. Methods, "Brute-force approach": "We also used a brute-force approach to prove that no program shorter than 17 instructions exists for sort 3. We had to enumerate roughly 10^{32} programs and, even with pruning heuristics, it took more than 3 days to prove this hypothesis. For sort 4 and above this approach is infeasible." The paper gives neither the exact search space nor a certificate; it makes no optimality claim for sort4 or sort5.

kernel	L	load	store	`mov` r,r	`cmp`	`cmovl/le`	`cmovg/ge`	x86 regs
AlphaDev sort3 (repo)	17	3	3	2	3	2/2	2/0	4
AlphaDev sort4 (repo)	28	4	4	5	5	7/0	0/3	5
AlphaDev sort5 (SI, 42)	42	5	5	5	9	4/3	4/7	6
AlphaDev sort5 (repo, 43)	43	5	5	6	9	6/7	5/0	6
network sort3 (3 comparators)	18	3	3	3	3	0/0	6/0	4
network sort4 (5 comparators)	28	4	4	5	5	0/0	10/0	5
network sort5 (9 comparators)	46	5	5	9	9	0/0	18/0	6

Table 2: Instruction mix. The network rows are our reconstruction of the human benchmark: an optimal sorting network with 4 instructions per comparator (`mov t,a; cmp a,b; cmovg a,b; cmovg b,t`) plus n loads and n stores, i.e. $L = 2n + 4c$. It gives exactly the human lengths of Table 1a (18, 28, 46); [1] does not list these programs.

AlphaDev's search space. The actions are x86 instructions in AT&T syntax (main text and Extended Data Table 1 list `mov`, `cmp`, `cmov`X and jumps; VarInt also uses `add`). All 15 published fixed-sort kernels for $n \leq 5$ (12 in SI App. G, 3 in the repository) use only `mov`, `cmp`, `cmovl`, `cmovle`, `cmovg`, `cmovge`, with register operands for `cmp` and `cmov`; jumps and immediates appear only in the variable sorts. Methods lists action pruning rules: "Memory locations are always read in incremental order. Registers are allocated in incremental order. We cannot compare or conditionally move to a memory location (illegal). We can read and write to each memory location only once. We cannot use non-initialized registers (illegal). Do not perform consecutive compare instructions." SI App. F adds that a `cmov` is allowed only after a `cmp` (Assumption 1). The published kernels do not obey all of these rules: the repo sort3 reads the cells in the order 1, 2, 0; SI `Sort3_17` reads cell 2 twice; SI `Sort3_20` has two consecutive `cmp`s. So the rules were search heuristics, not part of the problem. *Our lower bounds do not assume any of them* (except the memory-operand rule, which defines the model; Section 3).

Correctness tests and the 0-1 principle. SI App. F shows that the 0-1 principle of sorting networks fails for cmov programs (a 26-instruction program sorts all 0/1 inputs of length 4 but not $(4, 3, 2, 1)$, SI Prop. 2), and proves a restricted version used only for sort6–sort8. For $n \leq 5$ AlphaDev tests all weak orders; we do the same (order-type lemma, Section 4).

3 The exact model and the counting

3.1 What AlphaDev counts

Length in [1] is the number of x86 instructions of the kernel. Every instruction counts 1: loads (`mov` memory→register), stores (`mov` register→memory), register moves, `cmp` and `cmov`X. We checked this on every published kernel: the instruction counts of the listings are exactly the reported lengths (17, 28, 42, and 43 for the libc++ sort5). The kernels of Table 1 have exactly n loads and n stores (Table 2); some SI variants load a cell twice (e.g. `Sort3_17`: 4 loads). The data are 32-bit `int`s in a memory buffer, sorted in place; the conditions `l`, `le`, `g`, `ge` are the *signed* ones. Ties are judged by value (the test inputs contain equal elements).

Instruction set. $I_0 = \{\texttt{mov}, \texttt{cmp}, \texttt{cmovl}, \texttt{cmovg}\}$ and $I_1 = I_0 \cup \{\texttt{cmovle}, \texttt{cmovge}\}$. AlphaDev's kernels need I_1: the repository sort3 and both sort5 kernels (42, 43) fail if `le`/`ge` are replaced

by `l/g`; SI `Sort3_17` and both sort4 kernels still sort after this replacement. A lower bound for AlphaDev must therefore hold in I_1. (The shortcut "on distinct inputs `cmovle` = `cmovl`" is false: a `cmp` of two registers that hold copies of one input gives `eq` also on distinct inputs.) Lengths 17/28/42 are reachable in I_0 as well: I_0 kernels of these lengths exist for $n = 3, 4$ (above) and for $n = 5$ (found by our search, `programs/found/upper_sort5_I0_L42_mem.s`).

Flags. `cmovl` reads $SF \neq OF$, `cmovle` reads $ZF \vee SF \neq OF$, `cmovg` reads $\neg ZF \wedge SF = OF$, `cmovge` reads $SF = OF$. For these four conditions every one of the 16 states of (CF, ZF, SF, OF) acts as one of three abstract states `lt`, `eq`, `gt`; the state $ZF = 1, SF \neq OF$ acts as `lt` (Lean theorem `xeval_eq`; also checked on hardware).

3.2 The memory model MM

- Memory cells $m_0, \ldots, m_{n-1}$ hold the inputs $x_0, \ldots, x_{n-1}$ at the start. At the end m_j must hold the j-th smallest input.
- Registers $r_0, r_1, \ldots$: unbounded in number, arbitrary (garbage) values at the start; the flags are arbitrary at the start.
- Instructions: `mov r,[m]` (load), `mov [m],r` (store), `mov r,r'`, `cmp r,r'`, `cmov`X `r,r'`, X from the instruction set.
- **MM-1**: memory is accessed only by loads and stores. This is AlphaDev's rule "We cannot compare or conditionally move to a memory location (illegal)" (Methods). The x86 ISA also has memory *source* operands (`cmp r,[m]`, `cmp [m],r`, `cmov`X `r,[m]`); no published AlphaDev kernel uses them. We treat MM-1 as part of the model and study the variant without it separately (MM+, below).
- Correct = sorts every $x \in \mathbb{Z}_{32}^n$ for every initial register and flag state. Length = number of instructions.

We do not assume AlphaDev's search heuristics (cells read in order and only once, registers allocated in order, no reads of uninitialised registers, no `cmov` before the first `cmp`, no consecutive `cmp`s). A lower bound in MM therefore covers every program AlphaDev's search could produce, and more.

The question. In MM with I_1 and unbounded registers, does a kernel of length 16, 27 or 41 sort 3, 4 or 5 inputs? We show that none exists for 3 and 4 (Theorem 2), so AlphaDev's lengths 17 and 28 are optimal; for 5 the question stays open (Section 7).

3.3 From memory to registers: the snapshot model

Search tools work on registers only. The natural register model puts x_i in register P_i and asks for the sorted values in $P_0, \ldots, P_{n-1}$ at the end (model RF, "fixed outputs"). RF is *too strict*: a store can copy any register to memory at any time, so AlphaDev's model lets each output come from any register at any step. A lower bound in RF therefore does not bound MM. The exact register model is the *snapshot model* RS: as RF, plus a free pseudo-instruction `out j, r` that records the current value of r as output j (the last `out j` counts). Its cost k counts `mov`, `cmp` and `cmov` only.

Lemma 1 (Snapshot lemma)**.** *Under MM-1, with unbounded registers, in the strict and in the relaxed semantics:* $L_{\mathrm{MM}}^{\min}(n) = 2n + k_{\mathrm{RS}}^{\min}(n)$.

Proof sketch (full proof: Lemma L5 in the appendix). ($\leq$) Load every x_i into a register first; replace each `out j, r` by a store at the same position. ($\geq$) Every cell is loaded before its first store (else x_i is lost) and stored at least once (else output j is x_j, wrong for $n \geq 2$). For each cell, the last load before the first store is deleted and its target register renamed to P_i for its live range; every store becomes an `out`, plus a copy into a fresh register per segment if the cell is read again, and in each such segment the last load is deleted and renamed to that copy. Per cell this saves one load (before the first store) and one store (the final store, or the last reload after it). Cost $\leq L - 2n$. $\square$
The construction is implemented (`agents/spec/tools/ad2out.py`); it maps all 15 published kernels with $n \leq 5$ to RS programs of cost $\leq L - 2n$ (sort3: 11, sort4: 20, sort5: 32), and each image passes the strict judge. Hence the targets in RS are: no program of cost ≤ 10, ≤ 19, ≤ 31 for $n = 3, 4, 5$, in I_1.

Fixed outputs at the end. Let RE be RS with all outputs taken at the end (any registers). Always $k_{\mathrm{RS}} \leq k_{\mathrm{RE}} \leq k_{\mathrm{RF}}$. All published kernels still sort when their stores are moved to the end, so $k_{\mathrm{RE}} \leq L_{\mathrm{AD}} - 2n$, and a lower bound $k_{\mathrm{RS}} \geq L_{\mathrm{AD}} - 2n$ gives $k_{\mathrm{RS}} = k_{\mathrm{RE}}$. For $n = 2$: $k_{\mathrm{RS}} = k_{\mathrm{RE}} = 4$ (exhaustive search without pruning), so $L^{\min}_{\mathrm{MM}}(2) = 8$; for $n = 3$: 11. We do not need, and do not claim, $k_{\mathrm{RS}} = k_{\mathrm{RE}}$ for all n.

3.4 The variant MM+: memory source operands

MM+ is MM without MM-1: `cmp r,[m]`, `cmp [m],r` and `cmov`X `r,[m]` are allowed (all are x86-legal; there is no form with two memory operands and no `cmov` with a memory destination). Cells are still written only by stores; every instruction counts 1. The snapshot lemma fails in MM+: a `cmp` or `cmov` may read a cell after its last load (so the load cannot be renamed away), a final store cannot be removed if the cell is read later, and x_i can reach a register by a `cmov` from memory alone. In general only $L^{\min}_{\mathrm{MM+}} \geq k^{\min}_{\mathrm{RS}}$ holds. MM+ is strictly stronger than MM: for $n = 2$ a 7-instruction kernel exists (`mov r0,[m0]; cmp r0,[m1]; mov r1,[m1]; cmovg r1,[m0]; cmovg r0,[m1]; mov [m0],r0; mov [m1],r1`), against 8 in MM; and our search found MM+ kernels of length 15, 26, 39 for $n = 3, 4, 5$, all checked in the strict judge (Section 7.1). So AlphaDev's lengths are optimal only *because of* its rule MM-1; under AlphaDev's counting, the full x86 ISA allows shorter kernels.

model	definition	use
MM	memory via loads/stores only (MM-1), all instructions count	main claim (AlphaDev's setting)
RS	registers, free snapshot outputs; $L_{\mathrm{MM}} = 2n + k_{\mathrm{RS}}$	search for the main claim
RE, RF	outputs at the end (any / fixed registers)	not a bound for MM
MM+	MM plus memory source operands in `cmp`, `cmov`	side result: 15 / 26 / 40

Table 3: Models. Lower bounds transfer only from RS to MM (Lemma 1).

Main results.

Theorem 2. *In MM with I_1 and unbounded registers, no program of length $\leq L - 1$ sorts all $x \in \mathbb{Z}^n_{32}$ (for all garbage and flags), for $(n, L) = (3, 17)$ (certified: `cake_lpr`, Lean, Rocq) and $(4, 28)$ (exhaustive search; certificate checked in Lean up to $k \leq 17$). Hence AlphaDev's sort3 and sort4 kernels are shortest.*

For $n = 5$ the question stays open; Section 7 gives the evidence and the exact remaining case. Evidence that RF differs in practice: every AlphaDev kernel, with loads and stores removed, ends with its outputs in other registers (e.g. sort3 in P_0, S_{11}, P_1), so it is not an RF program of the same length.

4 The trust chain

link	statement	paper	Lean	machine check
X86	16 flag states $\to$ 3	App. A	`xeval_eq`	real hardware; certified ACL2 x86isa model
L2	strict $\Rightarrow$ T-relaxed	App.	`x86_to_relaxed`, `xsnap_to_relaxed`	–
L5	MM length $L \Rightarrow$ RS length $L - 2n$	App.	`l5`, `l5_strict`, `mm_lower_bound`	`l5map.py` on all 23 published programs
L3	$R = n + k$ registers suffice	App.	`register_bound_snap`	–
L1	order types (13/75/541); test subsets	App.	`x86_iff_reps`, `reps_card`	–
L6	symmetry rules keep a shortest program	App.	proved inside each verified checker (lean-reflect, lean-lrat, Rocq)	rules on/off agree ($n = 2, 3$); VeriPB + CakePB (canon, pcanon)
L4	exact-k sweep / NOP suffix	App.	`lower_bound_chain_snap`	–
enc	CNF = model instance	–	verified Lean encoder (lean-lrat)	2 more encoders; differential test (13 500 cases)
UNSAT	CNF has no model	–	Lean's LRAT checker	DRAT/LRAT $\to$ `cake_lpr`

Table 4: Trust chain. Every link has a paper proof (Appendix A); "Lean" names the formal theorem. The Lean x86 semantics agrees with real hardware (AMD EPYC 9654, hundreds of millions of runs) and with the certified ACL2 x86isa model (81 000 state vectors, 48 programs).

The most delicate link is the snapshot lemma (Lemma 1, Appendix L5). Its proof needs neither "loads first, stores last" nor AlphaDev's read-once/write-once rule: it handles reloads, spills and stores in the middle of the code, by one fresh register per store segment and by renaming the live range of the last load of each segment.

5 Search

Abstraction. By L1 a program is correct iff it is correct on one representative of each weak order type (13/75/541). For lower bounds any subset is enough (e.g. the $n!$ permutations, or a CEGIS set) — but permutations with I_0 are *not* a relaxation of weak orders with I_1 (a `cmp` of two copies of one input gives eq).

Relaxation. Scratch registers start as $T >$ all inputs, flags start eq (L2). This is sound; it can only make the relaxed optimum smaller (a T register is a known $+\infty$).

SAT encodings. `sat`: one-hot operands, CEGIS, NOP suffix, rules canon, pcanon, comm, dead, flags, cmpord, movfresh, snapcmov, mincmp with a rewriting (normal-form) proof. `sat2`: binary operands, sweep, rules with a lex-least proof.

Enumeration. `enum`: BFS over sets of distinct "columns" (Lemma S: with unbounded registers and snapshot outputs, a `mov` is only needed as half of a copy), exact deduplication, admissible heuristics, and the rule NG (normal form for strict programs; reviewed). `critic`: an independent register-level IDDFS.

Verified checkers. lean-reflect and Rocq check the enumerator's closure certificates (every successor of a kept state is kept or pruned by a proved rule, and no kept state is sorted); lean-lrat checks LRAT proofs of CNFs produced by an encoder that is proved correct in Lean.

Analytic bounds. #cmp $\geq \lceil \log_2 n! \rceil$ (3/5/7; tight for $n = 5$ with a 7-cmp kernel, so the sorting-network bound 9 does *not* hold); #cmov $\geq$ Huffman(n) = 5/8/12 (Kraft inequality on the paths of each output); for $n = 3$, #cmov ≥ 6, and for $n = 4$, #cmov ≥ 10, by exhaustive search in a relaxation.

6 Results for sort3 and sort4

n	claim (RS, I_1, T-relaxed, all weak orders)	tools	certificate	MM consequence
3	no program with $k \leq 10$	enum, critic, sat2, bounds agree; sat finds $k = 11$	`cake_lpr` (3 CNFs); Lean (kernel only; LRAT; reflection); Rocq	$L \geq 17$: AlphaDev optimal
4	no program with $k \leq 19$	enum: no strict program with $k \leq 19$ (1.39×10^9 states, rule NG); cross-checks: sat2 $k \leq 12$, Rocq $k \leq 14$, bounds (case $c = 9$)	123 GB closure certificate; Lean check up to $k \leq 17$	$L \geq 28$: AlphaDev optimal

Table 5: Lower bounds. Upper bounds: AlphaDev's kernels (checked in Lean on the x86 semantics for all inputs, garbage and flags, and on real hardware).

6.1 The sort3 bound: independent proofs and their trust bases

Theorem 3 (sort3)**.** *In AlphaDev's model $M_{AD}(3, I_1)$ — three signed 32-bit inputs in memory cells, sorted in place; instructions* `mov` *(load, store, register move),* `cmp`*,* `cmovl`*,* `cmovg`*,* `cmovle`*,* `cmovge`*;* `cmp` *and* `cmov` *on registers only (MM-1); unbounded registers; correct for every input, every initial register value and every initial flag state; every instruction counts — no program of length ≤ 16 sorts. AlphaDev's 17-instruction kernel is therefore optimal. Formal statements (Lean):* `LeanReflect.sort3_mm_lb : ∀ q, MMProg 3 q → q.length ≤ 16 → ¬ XCorrect 3 q`*, and the kernel-only* `KProto.D3k10I1G.lower_bound_kernel_x86` *for the register core.*

Every route below proves "no RS program with $k \leq 10$" ($n = 3$, I_1); the step to M_{AD} ($L \leq 16$) is Lemma 1 (Lean: `l5`, `mm_lower_bound`, kernel only).

route	search object	symmetry / pruning	proof checked by	trusted base	status
Lean, kernel only	closure certificate, set model + S_3 (2,215 states)	rules proved in Lean	Lean kernel	Lean kernel, 3 standard axioms	done ($k \leq 10$)
Lean, reflection	verified search, 6 perms (lean-reflect)	W, D, H1, polarity (proved in Lean)	Lean kernel	`native_decide` (Lean compiler)	done: `sort3_mm_lb`
Lean, set model	verified set-model search (lean-reflect)	Lemma S, G, W, P, D, H1, H3′ (proved in Lean)	Lean kernel	`native_decide`	done (`sort3_rs_lb_G`)
Lean, LRAT	verified CNF encoder (lean-lrat)	first-touch canon, normal form (proved in Lean)	Lean's LRAT checker	`native_decide`	done ($k \leq 10$; two builds)
SAT + LRAT (1)	CNF from `sat`, 6 perms	8 rules + cut `hcmp` (paper proofs, reviewed)	`cake_lpr`	Python encoder; rule proofs	done
SAT + LRAT (2)	same, no cut	8 rules (paper proofs, reviewed)	`cake_lpr`	same	done
SAT + LRAT (3)	second CNF (`sat2`, binary operands), CEGIS	R1–R6, M1–M4, live, C1 (reviewed)	`cake_lpr`	second encoder; rule proofs	done
Rocq	set-model checker; closed theorem by `native_compute`	Lemma S, W, P, G, D, H1, H2, E3, H3′ (proved in Rocq)	Rocq kernel	`native_compute`	done (`sort3_lb10`)
CakePB	lean-lrat CNF + canon/pcanon dominance + LRAT, one proof	certified in the same proof	`cake_pb_cnf`	CakeML/HOL4	done for $k \leq 8$
enumeration	set-model BFS (enum), register IDDFS (critic)	reviewed rules	–	C++ code	cross-check

Table 6: Proofs of $k_{\text{out}}(3) \geq 11$, i.e. Theorem 3. "`native_decide`" = trust in the Lean compiler. The Rocq and Lean set-model checkers accept the same certificate file. The upper bound (AlphaDev's kernel is an MM program of length 17 and sorts) is proved in Lean with the kernel only (`alphadev_mm`).

6.2 The sort4 bound

Theorem 4 (sort4)**.** *In $M_{AD}(4, I_1)$ with unbounded registers no program of length ≤ 27 sorts; AlphaDev's 28-instruction kernel is optimal. The same holds in I_0 (AlphaDev's sort4 kernel also sorts after replacing* `cmovle/cmovge` *by* `cmovl/cmovg`*).*

Evidence. The enumerator explores all strict-correct register cores with $k \leq 19$ on the 24 permutations (1.39×10^9 states, 96 minutes on 40 cores, peak 168 GB) and finds none. Its pruning rules, including the strict normal form NG, have paper proofs reviewed by two reviewers and by an independent model (Codex), and NG is also proved in Lean (`NGMain.lower_bound_strict`, standard axioms). The admissible heuristic was tested along every prefix of all 15 AlphaDev

register cores (0 violations). The closure certificate (123 GB, 16.7 GB compressed) is checked by the verified Lean checker up to $k \leq 17$ (`sort4_x_lb17`, 2.56 million states) and by the Rocq checker up to $k \leq 14$; checking the full $k \leq 19$ certificate needs a faster verified checker (about 12–40 hours estimated) and is left as future work. Independent partial results agree: `sat2` ($k \leq 12$, SAT) and the bounds tool (case $c = 9$, exact set model).

6.3 Latency: AlphaDev's kernels are not Pareto-optimal

Depth model: critical path over read-after-write edges on registers and flags (register renaming, so write-after-read and write-after-write do not count); `cmov` always reads its destination, its source and the flags; latencies from uops.info in three models: U (all 1), E (`mov` 0), Z2 (Zen 2: `mov` 0, `cmp`→`cmov` 2). Lower bounds use a *relaxed DAG*: each `cmp` a flag node, each `cmov` a select node, `mov` free; its depth is at most the program's depth, for any length and any output rule (proofs reviewed). Results (DRAT-checked UNSAT for the bounds, judge-checked programs for the constructions):

- minimum depth: $n = 3$: 4 (U, E), 5 (Z2); $n = 4$: 5 (U, E), 6 (Z2); $n = 5$: 5 or 6 (U, E; a depth-6 kernel with $k = 174$ exists, and depth ≥ 5 is DRAT-checked);
- $n = 3$: every program of depth ≤ 4 needs $k \geq 12$; a $k = 11$ program of depth 5 exists, so AlphaDev's sort3 $(k, \text{depth}) = (11, 6)$ is dominated;
- $n = 4$: a $k = 20$ kernel exists at depth 5;
- $n = 5$, $k = 32$: a kernel of depth U10/E10 beats AlphaDev's Sort5_42 (U13/E11/Z2 16) at the same length.

AlphaDev optimised length for the fixed sorts (it reports that length and latency correlate); our results show that at optimal length, depth can still be improved.

7 sort5: the search for a shorter kernel

The target is a strict-correct register core with $k \leq 31$, i.e. an MM kernel of length ≤ 41. A full lower-bound proof by the methods of Section 6 is out of reach: extrapolation from $n = 4$ gives at least 10^{15} abstract states. We therefore ran a large discovery effort (stochastic search, exact routing for fixed value skeletons, exhaustive searches in restricted program classes, LLM-guided program design) and recorded every negative result with its exact scope. Write a register core's cost as $k = c + g + v$ (`cmp`s, `cmov`s, `mov`s).

The known optima. We collected 274,035 correct programs with $k \leq 32$; all have $k = 32$ exactly and fall into 30 value-set classes. 27 classes have $c = 9$ and $g + v = 23$ (AlphaDev's split is $9+18+5$; we also found $9 + 21 + 2$); 3 classes are new and have $c = 8$ and $g + v = 24$ ($8 + 20 + 4$, $8 + 22 + 2$). So sort5 at AlphaDev's length is possible with 8 comparisons, and in every class we found, fewer comparisons cost exactly as many extra `cmov`s and copies. One $c = 9$ class uses values that are not the term set of any 9-comparator sorting network.

Two lemmas.

Proposition 5 (garbage never helps)**.** *Every strict-correct program can be transformed into a program that never reads a register before writing it, with no more instructions and no more* `cmp`*s.*

Proposition 6 (no lattice kernel)**.** *No sort5 register core with $k \leq 31$ exists in which every register always holds a min/max term of the inputs.*

Proposition 5 has a reviewed proof sketch and was tested on 5,606 programs (0 failures). Proposition 6 is an exhaustive breadth-first search (1.51×10^9 states, unbounded registers, reviewed tool, 9/9 positive controls on known $k = 32$ kernels). Together they show: a 41-instruction sort5 kernel, if it exists, must write at least one value that is not a min/max of the inputs (for example a "partner" carried along after a comparison).

Exclusions within stated scopes. No register core with $k \leq 31$ exists

- for any value skeleton of the 16 flag classes of the known optima with min/max operands (640 skeletons each), of the 177 nine- and 4,609 ten-comparator sorting networks, of 1,684 "recompute instead of copy" skeletons, of all 161 $c = 8$ skeletons of length 32, and of more than 30,000 further neighbour and non-lattice skeletons (exact routing, no register cap);
- using only the terms of any 9- or 10-comparator network, or of a known optimum plus one or two extra values (about 68,000 term pools, exact search);
- of the form "sort four inputs, then insert the fifth" (inserting needs ≥ 12 instructions, DRAT-checked);
- with $c \leq 8$ and all `cmp` operands min/max terms (exhaustive);
- within three to four changes of either of the two best 41-instruction near misses (wrong on 9 and 14 of the 541 input orders); no window of 4–7 instructions of these, of the first $c = 8$ kernel (up to 8), or of 17 further $c = 8$ kernels (4–6) can be replaced by a shorter one.

Stochastic search ($\approx 7 \times 10^9$ proposals), quality-diversity search and LLM-guided design found many 32-instruction cores and no 31.

The open case. What remains are garbage-free programs that write one or more non-min/max values. For one or two such values we built SAT encodings (reviewed as sound), but they decide none of the 1,929 cases within an hour, and an exact enumeration would need 10^{11}–10^{12} states; programs with three or more such values were not searched exhaustively. We therefore conjecture, but do not claim, that 42 is optimal for sort5.

7.1 Beyond AlphaDev's action space: memory operands

AlphaDev's action space excludes memory operands in `cmp` and `cmov` ("We cannot compare or conditionally move to a memory location (illegal)", Methods). The x86 ISA has them: `cmp r32,m32`, `cmp m32,r32` and `cmov`*cc* `r32,m32` (memory *source*; there is no form with a memory destination and none with two memory operands). Model MM+ allows exactly these forms and keeps everything else of MM, including the counting: every instruction counts 1, and a memory operand does not change that.

Shorter kernels. Table 7 lists MM+ kernels that are shorter than AlphaDev's for every n we studied. The kernels for $n = 3, 4, 5$ were found by stochastic search (MCMC with exhaustive window re-optimization); the $n = 2$ kernel is

`mov r0,[m0]; cmp r0,[m1]; mov r1,[m1]; cmovg r1,[m0]; cmovg r0,[m1]; mov [m0],r0; mov [m1],r1`.

Each kernel passes our strict reference judge (all weak orders, symbolic garbage registers and flags, random 32-bit inputs with `INT_MIN`, `INT_MAX` and ties), and the kernels for $n = 3, 4$ and a 40-instruction kernel for $n = 5$ sort on real hardware (AMD EPYC 9654, more than 3×10^6 runs each with garbage in all scratch registers and random flags, memory operands kept as memory operands; 0 mismatches against the judge semantics); the 39-instruction $n = 5$ kernel is checked by the strict judge and by an independent checker, but was not run on hardware.

n	MM (AlphaDev)	MM+	load	store	`mov` r,r	`cmp` r,r	`cmp` r,m	`cmov` r,r	`cmov` r,m
2	8	**7**	2	2	0	0	1	0	2
3	17	**15**	2	3	0	0	3	3	4
4	28	**26**	4	4	2	4	1	9	2
5	42	**39**	4	5	3	6	3	12	6

Table 7: Shortest known kernels with and without memory source operands. For $n = 2$ both columns are optimal (exhaustive search, Section 7.1), and for $n = 3$ the MM+ value 15 is optimal by search; for $n = 3, 4$ the MM column is optimal (Theorem 2), for $n = 5$ it is the best known; for $n = 4, 5$ the MM+ column is an upper bound.

Why memory operands help. A cell keeps its input until the kernel stores into it, so it acts as a free, read-only copy of x_i. In MM a kernel that needs x_i both as a value that a `cmov` will overwrite and as an unchanged value must spend a register move (or a second load) on a copy; in MM+ the unchanged value can be read from memory by `cmp r,[m]` or `cmov r,[m]` at no extra cost. The $n = 3$ kernel shows the effect most clearly: it has no register-to-register `mov` at all and only two loads; the third input enters registers only through `cmov`s from memory. The same property breaks the snapshot lemma (Lemma 1): a load can no longer be renamed away if a later `cmp` or `cmov` reads the cell, and a cell may never be loaded at all. In general only $L^{\min}_{\mathrm{MM+}} \geq k^{\min}_{\mathrm{RS}}$ holds, so the MM+ question needs its own search over states with cells.

Optimality in MM+. For $n = 2$ an exhaustive breadth-first search over exact abstract states (cells, multiset of register value vectors, flags; T-relaxed, all weak orders) gives $L^{\min}_{\mathrm{MM}}(2) = 8$ and $L^{\min}_{\mathrm{MM+}}(2) = 7$; the MM value agrees with Lemma 1 ($2n + k_{\mathrm{RS}} = 4 + 4$). For $n = 3$ an independent iterative-deepening search over the same exact states, with an admissible bound (cells off goal, missing goal values, and $\lceil \log_2 \rceil$ of the largest class of inputs not yet told apart), finds no MM+ kernel of length ≤ 14 (1.2×10^9 nodes, 35 minutes); so $L^{\min}_{\mathrm{MM+}}(3) = 15$. This is a search result without a certificate; the same search finds the known 15-instruction kernels, a check that its pruning keeps the optimum. A shortest MM+ sort3 even exists in I_0.

Shorter is not faster here. AlphaDev uses length as the cost because length and latency correlate for its kernels. With memory operands this link breaks: in our measurements on the AMD EPYC 9654 the MM+ kernels have a longer dependency-chain latency than AlphaDev's (median cycles per call, harness baseline subtracted: $n = 3$: 13.5 vs. 9.7; $n = 4$: 15.4 vs. 10.4; $n = 5$, 40-instruction kernel: 24.2 vs. 16.7) and no better throughput, because each memory operand puts a load on the critical path. (These timings come from a shared, loaded machine, so the absolute values are approximate.) So MM+ is a statement about AlphaDev's *cost model* (instruction count), not a faster sort.

Consequence for the main result. AlphaDev's lengths 17, 28 and 42 are optimal in AlphaDev's own action space (MM) for $n = 3, 4$ (and not beaten for $n = 5$), but not in the full x86 instruction set restricted to `mov`, `cmp` and `cmov`: the rule MM-1 is necessary for the claim. We state the main theorem for MM and report MM+ separately.

8 Threats to validity: what is outside the model

- **Memory operands (MM+).** x86 allows `cmp r,m`, `cmp m,r`, `cmovcc r,m`. With them, sort3, sort4 and sort5 fit in 15, 26 and 39 instructions (Section 7.1). So AlphaDev's kernels are optimal only under MM-1. Our snapshot lemma does not hold for MM+ (Lemma L8: only $L \geq k_{\mathrm{RS}}$); an MM+ optimum needs its own search.
- **Other instructions**: SIMD min/max, branches, arithmetic and masks, `setcc`, `lea`, unsigned conditions (they read flags that are not a function of the signed order). Adding `cmove`, `cmovne` and `xchg` gave no shorter kernel in our searches, and `xchg` is provably redundant (Lemma L11).
- **Other costs**: latency, throughput, code size (AlphaDev reports that length and latency correlate for fixed sorts).
- **Data**: signed int32. Unsigned or floating-point kernels use other opcodes.
- **Registers**: our lower bounds allow unbounded registers (stronger); real x86 has at most 14 free GPRs.
- **Trust base**: the Lean kernel (and, for some routes, the Lean compiler or the Rocq `native_compute` feature), `cake_lpr`, the Python encoders for the `cake_lpr` routes, and the agreement of the x86 semantics with real hardware and the ACL2 x86isa model. For sort4 the trusted base also includes the enumerator for $18 \leq k \leq 19$.
- **Hardware**: all runs used one AMD EPYC 9654; no Intel processor was tested.

9 Related work

AlphaDev. See Section 2. AlphaDev's only optimality statement is the brute-force claim for sort3 (about 10^{32} programs, with pruning, no certificate, search space not stated) [1]. The kernels went into LLVM libc++ (review D118029, landed 2022-04-08 as commit `194d1965d2c8`) [3], where they were accepted on benchmarks; the patch makes no optimality claim.

Ullrich and Hack. The closest work is "Synthesis of Sorting Kernels" [4] (CGO 2025). They search sorting kernels with A*/Dijkstra over abstract states, with optimality-preserving heuristics and optional non-optimality-preserving cuts. They enumerate all optimal kernels for $n = 3$ (length 11 without memory instructions, 5602 programs after symmetry reduction) and report that no kernel of length 19 exists for $n = 4$ (exhaustive, two weeks), a "new tight lower bound"; for $n = 5$ they find kernels of length about 33 with a cut, and give no lower bound. Table 8 compares their model with ours, point by point. Each point where their model is *more restrictive* than AlphaDev's (fewer programs) means that their lower bound excludes fewer programs and so does not imply a lower bound for AlphaDev's setting.

	Ullrich and Hack [4]	this work (MM via RS)
instructions	`mov`, `cmp`, `cmovl`, `cmovg` (I_0)	I_1 (adds `cmovle`, `cmovge`), as AlphaDev; I_0 as a side result
memory, counting	none; registers only (add $2n$ to compare)	loads and stores counted; exact map $L = 2n + k$ (Lemma 1)
outputs	fixed in $r_1..r_n$ at the end (RF)	any register at any step (RS) = AlphaDev's stores
scratch registers	a fixed number m; the paper does not state m; the artifact code uses one (`SWAPS = 1`)	unbounded ($R = n + k$ suffices, Lemma L3)
inputs tested	the $n!$ permutations (no ties)	all weak orders (ties); permutation subsets only inside sound relaxations
initial state	registers hold the inputs; reads of uninitialised scratch registers are forbidden	strict: garbage registers and flags (reads of uninitialised registers allowed); lower bound via a sound relaxation
pruning rules	argued in prose	each rule with a paper proof (and Lean where marked)
evidence	exhaustive-search runs, no proof log	DRAT/LRAT certificates checked by `cake_lpr`; verified checkers in Lean and Rocq; two independent search tools
results	$n = 3$: 11; $n = 4$: 20; $n = 5$: none	$n = 3$: 11 (certified); $n = 4$: 20 (exhaustive search, certificate checked to $k \leq 17$); $n = 5$: 32 best known, ≤ 31 open

Table 8: Ullrich and Hack's model versus ours. In their model a lower bound of 11 / 20 does not imply one for AlphaDev's setting: $I_0 \subsetneq I_1$, RF is stricter than RS, one scratch register is stricter than unbounded registers, and forbidding reads of uninitialised registers excludes programs that are correct on x86 (e.g. a 0-`mov` sort3 core with one such read, found by our search).

Their $n = 3$ and $n = 4$ values agree with ours. Their lower bound transfers to tie-correct kernels in I_0 with one scratch register, fixed outputs and no reads of uninitialised registers (distinct-input correctness is weaker than tie correctness), but not to I_1, to more scratch registers, or to snapshot outputs. They also report that SMT, CEGIS, SyGuS (cvc5), ILP, PDDL planners and STOKE do not solve $n = 4$, and that STOKE fails already for $n = 3$.

Shorter kernels outside the model. Neri [5] gives a 15-instruction branchless sort3 and a 14-instruction sort3 with a loop; both use `sbb`/`adc`, table lookups and indexed stores, so they are outside the `mov`/`cmp`/`cmov` model. Tunney [6] explains the AlphaDev saving as the deletion of one `mov` from three compare-and-swap blocks (her 17-instruction x86 kernel passes our judge; it needs `cmovge`) and shows a 14-instruction ARM64 kernel with the three-operand `csel`, also outside the model. Mimicry [7] computes a shuffle index instead of swapping: a 9-instruction SIMD sort3, and scalar kernels with indexed stores (sort3 17, sort4 27 for signed values); these use instructions outside our model and make speed claims only. None of them claims or refutes optimality in AlphaDev's model.

Superoptimization. Massalin's superoptimizer [8], STOKE [9], Lens [10] and Souper [11] find short or fast instruction sequences by enumeration, stochastic search or solvers; none of them reports an optimal sorting kernel, and STOKE fails on sort3 in the experiments of [4].

Optimal sorting networks. The comparator model is solved for small n with SAT and computer-checked certificates: 9 inputs need 25 comparators [12], with a Coq-verified checker of the certificate [13], and 11 and 12 inputs need 35 and 39 comparators, with an Isabelle-verified checker [14]. These results count comparators, not instructions, and the 0-1 principle holds there. For cmov code a comparator is not atomic: one `cmp` can feed `cmov`s that do not form a full swap (the swap move), the instruction count is not 4× comparators, and the 0-1 principle fails (SI App. F of [1]). Our method follows the same pattern (SAT encoding, symmetry breaking, verified certificate checking) for the instruction model.

10 Conclusion

In AlphaDev's own model, the 17-instruction sort3 kernel is optimal, with machine-checked certificates down to a kernel-only Lean theorem, and the 28-instruction sort4 kernel is optimal by exhaustive search. For sort5 we found no kernel shorter than 42, a new 8-comparison family of the same length, and two structural results that reduce the open question to kernels that carry non-min/max values. Closing that case needs a new idea, for example a lemma that bounds the number of such values in a shortest kernel. Two further observations matter for the original claim: AlphaDev's lengths are optimal only because its rules forbid memory operands, and at optimal length the critical path of its kernels can still be shortened.

A Soundness proofs

A.1 Machine, semantics, output conventions

Registers $\mathrm{Reg} = \{P_0, \dots, P_{n-1}\} \cup \{S_0, S_1, \dots\}$, a flag $F \in \{\mathsf{lt}, \mathsf{eq}, \mathsf{gt}\}$. Instructions (destination first): $\texttt{mov}\, D, S$; $\texttt{cmp}\, A, B$ sets F to the outcome of comparing A with B; $\texttt{cmov}c\, D, S$ for $c \in \{l, g, le, ge\}$ sets $D := S$ iff $F \in C(c)$ with $C(l) = \{\mathsf{lt}\}$, $C(g) = \{\mathsf{gt}\}$, $C(le) = \{\mathsf{lt}, \mathsf{eq}\}$, $C(ge) = \{\mathsf{gt}, \mathsf{eq}\}$. I_0 uses l, g; I_1 uses all four. Reads and writes: $W(\texttt{mov}\, D, S) = \{D\}$, $R(\texttt{mov}\, D, S) = \{S\}$; $W(\texttt{cmp}\, A, B) = \{F\}$, $R = \{A, B\}$; $W(\texttt{cmov}c\, D, S) = \{D\}$, $R = \{D, S, F\}$. Only `mov` is a *full definition* of its destination.

A run of $p = i_1 \cdots i_k$ from a state σ_0 gives states $\sigma_0, \dots, \sigma_k$ (position t = after i_t). *Strict* correctness quantifies over all inputs $x \in \mathbb{Z}_{32}^n$, all scratch values and all F. The real flags $(ZF, SF \oplus OF)$ have four states; the four conditions read them as lt iff $SF \neq OF$, eq iff $SF = OF \wedge ZF$, gt iff $SF = OF \wedge \neg ZF$, so the three-valued F is exact. *T-relaxed* correctness: inputs with $x_i <$ INT_MAX, all $S_j = T :=$ INT_MAX, $F = \mathsf{eq}$.

An RS program is a pair (p, o) with $o(j) = (r_j, t_j)$, $0 \le t_j \le k$; it is correct on x if $\sigma_{t_j}(r_j) = \mathrm{sort}_j(x)$ for all j. (RF: $o(j) = (P_j, k)$. RE: $t_j = k$.)

A.2 Order types (L1)

Let $\mathcal{W}_n$ be the weak orders on $\{0..n-1\}$ ($|\mathcal{W}_n| = 1, 3, 13, 75, 541$ for $n = 1..5$). *Abstract run* on $w \in \mathcal{W}_n$: registers hold labels in $\{0..n-1\} \cup \{T\}$, compared by w, with T above every index and $\mathrm{cmp}(T, T) = \mathsf{eq}$.

Lemma 7 (L1). *Let $\tau(x) = w$ and $x_i <$ INT_MAX. In the T-relaxed run on x, at every position every register holds $v_x(\ell)$, where ℓ is its label in the abstract run on w, $v_x(i) = x_i$, $v_x(T) =$ INT_MAX; and the flags agree. Hence (p, o) is relaxed-correct on x iff it is abstractly correct on w.*

Proof. Induction on positions; the only nontrivial case is `cmp`: $v_x(\ell) < v_x(\ell')$ iff $\ell <_w \ell'$ because $\tau(x) = w$ and every $x_i <$ INT_MAX. For outputs: $v_x(\ell) = \mathrm{sort}_j(x)$ iff $\ell \neq T$ and $x_\ell = \mathrm{sort}_j(x)$, a property of w alone. □

Corollary 8 (L1b). *For $W \subseteq \mathcal{W}_n$: if no length-$k$ program is abstractly correct on W, none is relaxed-correct.*

A.3 Relaxation (L2) and registers (L3)

Lemma 9 (L2). *Strict correctness implies T-relaxed correctness (all output conventions).*

Proof. The relaxed run is the strict run with scratch = INT_MAX and the real flag state $ZF = 1$, $SF = OF$. □

Lemma 10 (L3). *If a relaxed-correct* RS *program of length k exists ($n \geq 2$), one exists over $P_0..P_{n-1}, S_0..S_{k-1}$.*

Proof. With no `cmp`, $F =$ `eq` throughout; l, g-cmovs are no-ops and le, ge-cmovs are `mov`s, so every label is independent of w, and output 0 fails on some w. Hence $\leq k-1$ instructions write registers. Never-written scratch registers hold T forever: merge them into one. Rename the at most k scratch names bijectively; all scratch registers start equal, so the run is unchanged. □

A.4 Memory model and the counting map (L5)

An MM program uses cells $m_0..m_{n-1}$ (holding x at the start, required to hold $\mathrm{sort}(x)$ at the end) and registers. Instructions: register instructions as above, *loads* $\texttt{mov}\, r, [m_i]$, *stores* $\texttt{mov}\, [m_i], r$. Assumption (MM-1): no other instruction touches a cell.

Lemma 11 (L5). *If an* MM *program q of length L is correct (strict, or T-relaxed), then an* RS *program of length $\leq L - 2n$ is correct in the same semantics. Conversely an* RS *program of length k yields an* MM *program of length $k + 2n$.*

Proof. (a) Each cell is loaded before it is first stored to. Otherwise x_i never reaches a register (MM-1), the final memory does not depend on x_i, but $\mathrm{sort}(x)$ does. *(b) Each cell is stored to.* Otherwise m_j ends as $x_j \neq \mathrm{sort}_j(x)$ for some x.

Segments. The stores to m_i split time into segments $s = 0, 1, \ldots, e_i$; segment 0 is before the first store and has value x_i; segment $s \geq 1$ starts at the s-th store. By (a) segment 0 contains a load; by (b) $e_i \geq 1$. Let $G_{i,0} := P_i$ and let $G_{i,s}$ ($s \geq 1$) be fresh registers.

Live ranges. A load or `mov` into r at time t starts a range of r that ends just before the next load or `mov` into r (or at the end). A `cmov` into r stays in the current range.

Translation $q \mapsto (p, o)$, instruction by instruction, with a name map ν (initially identity on registers):

- register instruction: emit it with operands renamed by ν;
- load $\texttt{mov}\, r, [m_i]$ in segment s: if it is not the last load of the segment, emit $\texttt{mov}\, \nu'(r), G_{i,s}$ with $\nu'(r)$ a fresh name, set $\nu(r) := \nu'(r)$; if it is the last load, emit nothing and set $\nu(r) := G_{i,s}$;

- store $\texttt{mov}\,[m_i], r$ that opens segment s: if the segment has a load, emit $\texttt{mov}\,G_{i,s}, \nu(r)$; otherwise emit nothing. If $s = e_i$, set $o(i) := (\nu(r), \text{current position})$.

Invariant. After translating a prefix of q (same semantics, corresponding initial states; fresh names start as garbage, resp. T): (I1) for every register r, the value of r in q equals the value of $\nu(r)$ in p whenever r's current range is read later; (I2) for every cell i in segment s whose remaining loads are not yet all done, $G_{i,s}$ in p holds the segment value; (I3) the flags agree. Each step keeps the invariant: register instructions act on corresponding values (I1, I3); a non-last load copies the segment value by (I2); at the last load, $G_{i,s}$ holds the segment value and is not read again except through $\nu(r)$ (no later load in this segment; later segments use other names), so (I1) holds for the new range; a store with a later load writes the segment value to $G_{i,s}$ (I2). Two ranges never share a name: fresh names are fresh, and $G_{i,s}$ is renamed into at most one range (the last load), after which nothing else reads or writes $G_{i,s}$. At the final store of m_i, (I1) gives $\sigma(\nu(r)) =$ the value stored, which is the final content of m_i, i.e. $\text{sort}_i(x)$. So (p, o) is correct.

Count. Per cell: segment 0 loses its last load (-1); segment $e_i \geq 1$ loses its store (no load) or its last load (-1). Every other emitted instruction replaces one instruction of q. Hence $|p| \leq L - 2n$.

Converse. Emit $\texttt{mov}\,P_i, [m_i]$ for all i, then p, inserting $\texttt{mov}\,[m_j], r_j$ at position t_j. Cells are not read after the first n instructions, so the stores are safe. □

Remark 12. (i) The proof needs neither "loads first / stores last" nor AlphaDev's read-once/write-once rule. (ii) Fixing both inputs and outputs (RF) is *not* implied: MM chooses load targets and store sources freely. (iii) Without MM-1 (memory sources in `cmp`/`cmov`) a load can merge into a compare and (a) gives no load per cell; the lemma then fails as stated.

A.5 Padding (L4)

Lemma 13 (L4)**.** *(a) Without symmetry rules, UNSAT at exact length k implies UNSAT for all $k' \leq k$ (append $\texttt{mov}\,P_0, P_0$). (b) With the rules of L6, use either the sweep "UNSAT for every $k' \leq K$" or a `nop` opcode allowed only as a suffix (rules apply to the non-`nop` prefix).*

Proof. (b) If a correct program of length $\leq K$ exists, a shortest one has length $m^* \leq K$ and by Lemma A.6 a rule-satisfying one of length m^* exists. □

A.6 Symmetry breaking and pruning (L6)

Fix a total order $\prec$ on instructions (lexicographic in opcode, then operands, with a fixed register order) and extend it lexicographically to programs of equal length (snapshot lists break ties). Operations on relaxed-correct RS programs of length k (each preserves length and correctness): ρ_π permutes scratch names (all start as T); ι_π permutes P names, also in o (the program then sorts $x \circ \pi$, and $\text{sort}(x \circ \pi) = \text{sort}(x)$; not valid in RF); σ_t swaps i_t, i_{t+1} if they are *independent*: $W(i_t) \cap (R(i_{t+1}) \cup W(i_{t+1})) = \emptyset = W(i_{t+1}) \cap R(i_t)$, and moves a snapshot at position t reading r to $t+1$ if $r \in W(i_t)$, to $t-1$ if $r \in W(i_{t+1})$; ϕ_t replaces $\texttt{cmp}\,A, B$ at t by $\texttt{cmp}\,B, A$ and swaps $l \leftrightarrow g$, $le \leftrightarrow ge$ in the cmovs up to the next `cmp`. Let μ replace every le/ge-cmov before the first `cmp` by `mov` (relaxed: $F =$ `eq` there).

Lemma 14 (Master lemma)**.** *Let p be a shortest relaxed-correct program and $p^* = \min_\prec C(\mu(p))$, where C is the closure under $\rho, \iota, \sigma, \phi$. Then p^* is shortest and correct, has no le/ge-cmov before the first `cmp`, and $g(p^*) \not\prec p^*$ for every applicable operation g.*

Proof. $C(\mu(p))$ is finite and every member is correct of length $|p|$. No operation creates a pre-`cmp` cmov: σ never swaps a cmov with a `cmp` (both use F), ρ, ι keep opcodes, ϕ acts after a `cmp`. □

Consequences (each is "some g would give a $\prec$-smaller program"): R1 scratch names first occur in the order $S_0, S_1, \ldots$ in the scan order of $\prec$; R2 independent neighbours satisfy $i_t \prec i_{t+1}$ (identical instructions are never independent); R5 every $\texttt{cmp}\, A, B$ has $A \prec B$; R6 (RS/RE) P names first occur in order; R7 no le/ge-cmov before the first `cmp`. Minimality rules (every shortest program satisfies them, since deleting the instruction keeps correctness): S1 no $\texttt{mov}/\texttt{cmov}\, D, D$; S2 no dead write ($D$ not read — as a source, as a cmov destination, by a `cmp`, or by a snapshot — before the next full definition of D or the end); S3 every `cmp` is followed by a cmov before the next `cmp`/end; S4 no l/g-cmov before the first `cmp`; S5 no $\texttt{cmp}\, A, A$. Implied constraints (true for every correct program, e.g. "a value needed by a later snapshot is held by some register") combine with all rules.

Theorem 15 (Joint soundness)**.** *If a relaxed-correct* RS *program of length $\leq K$ exists, then for m^* = the least such length, a relaxed-correct* RS *program of length m^* over $n + m^*$ registers satisfies R1, R2, R5, R6, R7, S1–S5 and every implied constraint.*

Proof. L3, then the master lemma applied to a shortest program over $n+m^*$ registers; p^* is shortest, so S1–S5 hold. □

A.7 Comparison count (L7)

Lemma 16 (L7)**.** *A program correct on all $n!$ permutations (T-relaxed, any output convention, I_0 or I_1) has at least $\lceil \log_2 n! \rceil$ `cmp` instructions.*

Proof. Permutations with equal flag histories have equal labels in all registers (induction). So a `cmp` compares one label pair on a whole class: equal labels or T against anything give one outcome; distinct indices give `lt` or `gt`. Each `cmp` at most doubles the number of classes. At the end, snapshot j holds label $\pi^{-1}(j)$, so the $n!$ permutations lie in distinct classes. □

A.8 Main theorem

Theorem 17. *Fix n, $I \in \{I_0, I_1\}$, MM-1, AlphaDev counting (every instruction, incl. loads and stores), unbounded registers. If for every $k \leq L - 1 - 2n$ the encoding of "a relaxed* RS *program of length k over $n + k$ registers, correct on $W \subseteq \mathcal{W}_n$, satisfying a subset of the rules" is unsatisfiable, then no* MM *program of length $\leq L - 1$ sorts all $x \in \mathbb{Z}_{32}^n$ for all garbage and flags.*

Proof. L2, L5, joint soundness, L1b. □

References

[1] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature* 618(7964):257–263, 2023. DOI 10.1038/s41586-023-06004-9. Supplementary Information: 41586_2023_6004_MOESM1_ESM.pdf.

[2] Google DeepMind. AlphaDev repository, `sort_functions_test.cc`. `https://github.com/google-deepmind/alphadev`, accessed 2026-10-24.

[3] M. Gelmi. Introduce branchless sorting functions for sort3, sort4 and sort5. LLVM review D118029, `https://reviews.llvm.org/D118029`, landed 2022-04-08 (commit 194d1965d2c8).

[4] M. Ullrich, S. Hack. Synthesis of sorting kernels. In *Proc. CGO 2025*, pp. 1–14. DOI 10.1145/3696443.3708954. Artifact: Zenodo 10.5281/zenodo.14092980.

[5] C. Neri. Shorter and faster than Sort3AlphaDev. arXiv:2307.14503, 2023.

[6] J. Tunney. Understanding DeepMind's sorting algorithm. `https://justine.lol/sorting/`, 2023-06-12 (read via web.archive.org).

[7] Mimicry. Faster sorting beyond DeepMind's AlphaDev. `https://www.mimicry.ai/faster-sorting-beyond-deepminds-alphadev`, 2023 (read via web.archive.org).

[8] H. Massalin. Superoptimizer: a look at the smallest program. In *Proc. ASPLOS 1987*, pp. 122–126.

[9] E. Schkufza, R. Sharma, A. Aiken. Stochastic superoptimization. In *Proc. ASPLOS 2013*, pp. 305–316.

[10] P. M. Phothilimthana, A. Thakur, R. Bodik, D. Dhurjati. Scaling up superoptimization. In *Proc. ASPLOS 2016*. DOI 10.1145/2872362.2872387.

[11] R. Sasnauskas, Y. Chen, P. Collingbourne, J. Ketema, G. Lup, J. Taneja, J. Regehr. Souper: a synthesizing superoptimizer. arXiv:1711.04422, 2017.

[12] M. Codish, L. Cruz-Filipe, M. Frank, P. Schneider-Kamp. Sorting nine inputs requires twenty-five comparisons. *J. Comput. Syst. Sci.* 82(3):551–563, 2016.

[13] L. Cruz-Filipe, K. S. Larsen, P. Schneider-Kamp. Formally proving size optimality of sorting networks. *J. Autom. Reasoning* 59(4):425–454, 2017.

[14] J. Harder. An answer to the Bose-Nelson sorting problem for 11 and 12 channels. arXiv:2012.04400, 2020.